

Dünya Geneline Trafik Kameraları ve Hız Tuzakları Veritabanı

Expo & Supabase Entegrasyon Kılavuzu

Ücretsiz ve Açık Kaynak Veri Kaynakları
API Entegrasyonları & Kod Örnekleri
Supabase Veritabanı Şeması

2025

İçindekiler

Not: İçindekiler tablosu otomatik olarak oluşturulmuştur. Sayfa numaralarını güncellemek için tabloya sağ tıklayıp "Alanı Güncelle" seçeneğini kullanın.

1. Genel Bakış

Bu kılavuz, dünya genelinde trafik kameraları ve hız tuzakları verilerini ücretsiz olarak toplamak ve uygulamanızda kullanmak için kapsamlı bir kaynak sunmaktadır. Expo/React Native ile Supabase entegrasyonu için pratik kod örnekleri ve en iyi uygulamalar içermektedir. Mobil uygulamanızın iOS ve Android platformlarında yayınlanması hedeflenmektedir ve bu belge, gerçek zamanlı trafik kamera verilerine erişim sağlamanıza yardımcı olacaktır.

Trafik kameraları ve hız tuzakları verileri, sürücülerin güvenli sürüş yapmalarına yardımcı olmak ve trafik cezalarından kaçınmak için kritik öneme sahiptir. Bu veriler, farklı kaynaklardan toplanarak birleştirilebilir ve kullanıcılara gerçek zamanlı uyarılar sunmak için kullanılabilir. Aşağıda, bu verileri ücretsiz olarak edinmenin en etkili yollarını detaylı olarak açıklanmaktadır.

2. Ücretsiz Veri Kaynakları

Aşağıda, dünya genelinde trafik kameraları ve hız tuzakları verilerine erişim sağlayan ücretsiz ve açık kaynak platformlar listelenmiştir. Her bir kaynağın avantajları, dezavantajları ve kullanım yöntemleri detaylı olarak açıklanmaktadır.

2.1 OpenStreetMap (OSM) - Overpass API

OpenStreetMap, dünyanın en büyük açık haritalama projesidir ve topluluk tarafından sürdürülen kapsamlı trafik kameraları veritabanına sahiptir. Overpass API, OSM verilerini sorgulamak için kullanılan güçlü bir API'dir ve belirli etiketlere göre veri filtrelemeye olanak tanır. Bu kaynak tamamen ücretsizdir ve herhangi bir API anahtarı gerektirmez, ancak kullanım sınırlamaları bulunmaktadır.

Avantajlar:

- Tamamen ücretsiz ve açık kaynak
- Dünya genelinde kapsam
- API anahtarı gerektirmez
- JSON formatında çıktı
- Düzenli topluluk güncellemeleri

Dezavantajlar:

- Rate limiting (dakikada maksimum istek sayısı)
- Veri kalitesi bölgeden bölgeye değişebilir
- Mobil kameralar genellikle dahil değildir

Overpass API Örnek Sorgu:

```
[out:json][timeout:25]; ( node["highway"]="speed_camera" ({{bbox}});
node["enforcement"]="maxspeed" ({{bbox}});
node["enforcement"]="traffic_signals" ({{bbox}}); ); out body;
```

OSM Wiki'de hız kameraları için kullanılan etiketler detaylı olarak açıklanmaktadır.

highway=speed_camera etiketi sabit hız kameralarını, enforcement=maxspeed ise hız denetim noktalarını belirtir. Bu etiketleri kullanarak dünya genelindeki tüm kameraları çekebilirsiniz.

2.2 Lufop API

Lufop, Avrupa'nın en kapsamlı ücretsiz hız kamerası veritabanlarından biridir. 20'den fazla ülkede aktif kameraların konumlarını sunmaktadır. Topluluk tarafından güncellenen bu veritabanı, aylık

olarak test edilmekte ve doğrulanmaktadır. API erişimi tamamen ücretsizdir ve GPS uygulamaları için ideal bir kaynaktır.

Kapsanan Ülkeler:

- Fransa, İngiltere, Belçika, Hollanda
- Almanya, İspanya, İtalya, Portekiz
- İsviçre, Avusturya, Polonya, Çekya
- İsveç, Norveç, Danimarka, Finlandiya

Kamera Türleri:

- Sabit hız kameraları (Fixed speed cameras)
- Kırmızı ışık kameraları (Red light cameras)
- Ortalama hız ölçüm noktaları (Average speed)
- Mobil kamera bölgeleri (Mobile zones)
- İnşaat alanı kameraları (Construction zone)

2.3 Hükümet Açık Veri Portalları

Birçok ülke, trafik kameraları ve hız tuzakları verilerini resmi açık veri portalları üzerinden paylaşmaktadır. Bu kaynaklar genellikle en doğru ve güncel verileri sağlar, çünkü doğrudan yetkili kurumlar tarafından yayınlanmaktadır. Ancak her ülkenin veri formatı ve erişim yöntemi farklı olabilir ve bu nedenle entegrasyon daha karmaşık olabilmektedir.

Ülke	Portal	Veri Türü
ABD	data.gov	Trafik kameraları
Avustralya	data.act.gov.au	Hız kameraları
Kanada	ouvert.canada.ca	Trafik kameraları
Hindistan	data.gov.in	Kameralar
İngiltere	data.gov.uk	Hız kameraları
Avustralya (VIC)	vic.gov.au	Yol güvenliği kameraları

Tablo 1: Hükümet Açık Veri Portalları

2.4 GitHub Açık Kaynak Projeleri

GitHub üzerinde topluluk tarafından geliştirilen çeşitli projeler, hız kamerası verilerini toplamak ve dağıtmak için kullanılabilir. Bu projeler genellikle birden fazla kaynaktan veri birleştirir ve düzenli olarak güncellenir. Açık kaynak olması sayesinde projeleri kendi ihtiyaçlarınıza göre özelleştirebilirsiniz.

Önerilen Projeler:

1. Open-GATSO-POI: Avrupa hız kameraları için günlük güncellenen veritabanı
2. catchcam: Offline hız kamerası dedektörü için veritabanı
3. waze_traffic_api: Waze verilerinden polis ve kamera raporları

3. Supabase Veritabanı Şeması

Supabase, PostgreSQL tabanlı bir Backend-as-a-Service platformudur ve gerçek zamanlı veri senkronizasyonu, yerel JSON desteği ve güçlü sorgulama yetenekleri sunar. Aşağıda, trafik kameraları

verileri için optimize edilmiş bir veritabanı şeması önerisi sunulmaktadır. Bu şema, farklı kaynaklardan gelen verileri standartlaştırmak ve verimli sorgulama yapmak için tasarlanmıştır.

3.1 Ana Tablolar

```
-- Ülkeler tablosuCREATE TABLE countries ( id SERIAL PRIMARY KEY, iso_code VARCHAR(3)
UNIQUE NOT NULL, name VARCHAR(100) NOT NULL, created_at TIMESTAMP DEFAULT NOW());--
Kamera türleriCREATE TYPE camera_type AS ENUM ( 'speed_fixed', -- Sabit hız
kamarası 'speed_mobile', -- Mobil hız kamarası 'red_light', -- Kırmızı
ışık kamarası 'speed_average', -- Ortalama hız ölçüm 'traffic_light', --
Trafik ışığı kamarası 'construction', -- İnşaat alanı 'police' --
Polis kontrol noktası);-- Ana kameralar tablosuCREATE TABLE traffic_cameras ( id UUID
PRIMARY KEY DEFAULT gen_random_uuid(), latitude DECIMAL(10, 8) NOT NULL, longitude
DECIMAL(11, 8) NOT NULL, camera_type camera_type NOT NULL, speed_limit INTEGER,
road_name VARCHAR(255), direction INTEGER, country_id INTEGER REFERENCES
countries(id), source VARCHAR(50) NOT NULL, source_id VARCHAR(100), is_active
BOOLEAN DEFAULT true, verified BOOLEAN DEFAULT false, last_verified_at TIMESTAMP,
created_at TIMESTAMP DEFAULT NOW(), updated_at TIMESTAMP DEFAULT NOW(), -- Konum
indeksi CONSTRAINT valid_coordinates CHECK (latitude BETWEEN -90 AND 90 AND
longitude BETWEEN -180 AND 180));-- Konum tabanlı sorgular için GIST indeksiCREATE
INDEX idx_cameras_location ON traffic_cameras USING GIST ( point(longitude,
latitude) );
```

3.2 Ek Tablolar ve İndeksler

```
-- Kamera güncelleme loglarıCREATE TABLE camera_update_logs ( id SERIAL PRIMARY KEY,
camera_id UUID REFERENCES traffic_cameras(id), action VARCHAR(20) NOT NULL, old_data
JSONB, new_data JSONB, changed_by VARCHAR(100), created_at TIMESTAMP DEFAULT
NOW());-- Kullanıcı raporlarıCREATE TABLE user_reports ( id UUID PRIMARY KEY DEFAULT
gen_random_uuid(), camera_id UUID REFERENCES traffic_cameras(id), user_id UUID,
report_type VARCHAR(50) NOT NULL, description TEXT, status VARCHAR(20) DEFAULT
'pending', created_at TIMESTAMP DEFAULT NOW());-- Veri kaynağı meta bilgileriCREATE
TABLE data_sources ( id SERIAL PRIMARY KEY, name VARCHAR(100) NOT NULL, url
VARCHAR(500), api_endpoint VARCHAR(500), update_frequency VARCHAR(50), last_sync
TIMESTAMP, is_active BOOLEAN DEFAULT true);-- Performans için ek indekslerCREATE INDEX
idx_cameras_country ON traffic_cameras(country_id);CREATE INDEX idx_cameras_type ON
traffic_cameras(camera_type);CREATE INDEX idx_cameras_active ON
traffic_cameras(is_active);CREATE INDEX idx_cameras_source ON traffic_cameras(source);
```

4. Expo/React Native Entegrasyonu

Expo, React Native uygulaması geliştirmek için güçlü bir framework'tür ve Supabase ile sorunsuz entegrasyon sağlar. Aşağıda, uygulamanızda kullanabileceğiniz kod örnekleri ve best practice'ler sunulmaktadır. Bu örnekler, veri çekme, önbelleğe alma ve gerçek zamanlı güncellemeler için optimize edilmiştir.

4.1 Supabase Client Kurulumu

```
// lib/supabase.tsimport { createClient } from '@supabase/supabase-js';import
AsyncStorage from '@react-native-async-storage/async-storage';const supabaseUrl =
process.env.EXPO_PUBLIC_SUPABASE_URL!;const supabaseAnonKey =
process.env.EXPO_PUBLIC_SUPABASE_ANON_KEY!;export const supabase = createClient(
supabaseUrl, supabaseAnonKey, { auth: { storage: AsyncStorage,
autoRefreshToken: true, persistSession: true, detectSessionInUrl: false,
}, });
```

4.2 Kamera Verisi Çekme Servisi

```
// services/cameraService.tsimport { supabase } from '../lib/supabase';export interface
Camera { id: string; latitude: number; longitude: number; camera_type: string;
speed_limit: number | null; road_name: string | null; direction: number | null;
is_active: boolean; distance?: number;}export class CameraService { // Belirli bir
bölgedeki kameraları getir static async getCamerasInBounds( minLat: number,
maxLat: number, minLng: number, maxLng: number, types?: string[] ):
Promise<Camera[]> { let query = supabase .from('traffic_cameras')
```

```
.select('*')
    .eq('is_active', true)
    .gte('latitude', minLat)
    .lte('latitude', maxLat)
    .gte('longitude', minLng)
    .lte('longitude', maxLng);
if (types && types.length > 0) {
    query = query.in('camera_type', types);
}
const { data, error } = await query;
if (error) throw error;
return data;
} //
Kullanıcı konumuna yakın kameraları getir static async getNearbyCameras( lat:
number, lng: number, radiusKm: number = 5 ): Promise<Camera[]> {
const {
data, error } = await supabase.rpc('get_nearby_cameras', {
    user_lat: lat,
    user_lng: lng,
    radius_km: radiusKm
});
if (error) throw error;
return
data;
}}
```

4.3 PostgreSQL RPC Fonksiyonu

Supabase'de yakın kameraları getirmek için bir PostgreSQL fonksiyonu oluşturun:

```
CREATE OR REPLACE FUNCTION get_nearby_cameras( user_lat FLOAT, user_lng FLOAT, radius_km FLOAT DEFAULT
5) RETURNS TABLE ( id UUID, latitude FLOAT, longitude FLOAT, camera_type TEXT, speed_limit INTEGER,
road_name TEXT, distance_km FLOAT) AS $$BEGIN RETURN QUERY SELECT tc.id, tc.latitude::FLOAT,
tc.longitude::FLOAT, tc.camera_type::TEXT, tc.speed_limit, tc.road_name, ( 6371 * acos(
cos(radians(user_lat)) * cos(radians(tc.latitude)) *
radians(user_lng)) + sin(radians(user_lat)) * sin(radians(tc.latitude)) ) )::FLOAT
AS distance_km FROM traffic_cameras tc WHERE tc.is_active = true AND ( 6371 * acos(
cos(radians(user_lat)) * cos(radians(tc.latitude)) *
radians(user_lng)) + sin(radians(user_lat)) * sin(radians(tc.latitude)) ) ) <=
radius_km ORDER BY distance_km;END;$$ LANGUAGE plpgsql;
```

5. Veri Toplama Stratejisi

Dünya genelinde trafik kamerası verilerini toplamak için çok kaynaklı bir yaklaşım önerilmektedir. Bu strateji, verilerin doğruluğunu, güncelliğini ve kapsama alanını artırmak için tasarlanmıştır. Aşağıda, veri toplama sürecinin adım adım uygulaması açıklanmaktadır.

5.1 Overpass API'den Veri Çekme

```
// scripts/fetchOSMCameras.tsconst OVERPASS_API = 'https://overpass-api.de/api/interpreter';export async
function fetchOSMCameras(bounds: { minLat: number; maxLat: number; minLng: number; maxLng: number;}) {
const query = `
[out:json][timeout:60];
node["highway"="speed_camera"] (${bounds.minLat},${bounds.minLng},${bounds.maxLat},${bounds.maxLng});
node["enforcement"="maxspeed"] (${bounds.minLat},${bounds.minLng},${bounds.maxLat},${bounds.maxLng});
node["enforcement"="traffic_signals"] (${bounds.minLat},${bounds.minLng},${bounds.maxLat},${bounds.maxLng});
`;
const response = await fetch(OVERPASS_API, { method: 'POST', body:
`data=${encodeURIComponent(query)}` });
const data = await response.json();
return data.elements.map((el: any) => ({
source: 'osm',
source_id: `osm_${el.id}`,
latitude: el.lat,
longitude: el.lon,
camera_type: mapOSMType(el.tags),
speed_limit: el.tags?.['maxspeed'] ?
parseInt(el.tags.maxspeed) : null,
road_name: el.tags?.['name'] || null
}));
}function mapOSMType(tags: any): string {
if (tags.highway === 'speed_camera') return 'speed_camera';
if (tags.enforcement === 'traffic_signals') return 'red_light';
if (tags.enforcement === 'maxspeed') return
'speed_fixed';
return 'speed_fixed';
}
```

5.2 Lufop API Entegrasyonu

```
// scripts/fetchLufopCameras.tsconst LUFOP_API = 'https://api.lufop.net/v1/cameras';export async function
fetchLufopCameras(countryCode?: string) {
const params = new URLSearchParams();
if (countryCode) params.append('country', countryCode);
const response = await fetch(`${LUFOP_API}?${params}`);
const data = await response.json();
return data.cameras.map((cam: any) => ({
source: 'lufop',
source_id: `lufop_${cam.id}`,
latitude: cam.latitude,
longitude: cam.longitude,
camera_type:
mapLufopType(cam.type),
speed_limit: cam.speed_limit || null,
road_name: cam.road || null,
direction: cam.direction || null
}));
}function mapLufopType(type: string): string {
const typeMap: Record<string, string> = {
'FIXED': 'speed_fixed',
'MOBILE': 'speed_mobile',
'REDLIGHT':
'red_light',
'AVERAGE': 'speed_average',
'CONSTRUCTION': 'construction'
};
return typeMap[type] ||
'speed_fixed';
}
```

5.3 Veri Senkronizasyon İş Akışı

Veri senkronizasyonu için önerilen iş akışı aşağıdaki gibidir:

1. Her gün gece yarısı tam senkronizasyon çalıştırın
2. Kullanıcı konumuna göre önbellege alınmış veriyi sunun
3. Kullanıcı raporlarını gerçek zamanlı işleyin

4. Çakışan kayıtları otomatik birleştirin
5. Eski/inaktif kameraları düzenli olarak temizleyin

6. Performans Optimizasyonu

Mobil uygulamada yüksek performans sağlamak için veri çekme ve önbelleğe alma stratejileri kritik öneme sahiptir. Aşağıda, uygulamanızın hızlı ve verimli çalışması için önerilen optimizasyon teknikleri açıklanmaktadır.

6.1 Önbelleğe Alma Stratejisi

```
// hooks/useCachedCameras.tsimport { useQuery } from '@tanstack/react-query';import AsyncStorage from
'react-native-async-storage/async-storage';import { CameraService, Camera } from
'../services/cameraService';const CACHE_KEY = 'cached_cameras';const CACHE_DURATION = 24 * 60 * 60 * 1000;
// 24 saatexport function useCachedCameras( lat: number, lng: number, radiusKm: number = 10) { return
useQuery({ queryKey: ['cameras', lat, lng, radiusKm], queryFn: async () => { // Önce önbelleği
kontrol et const cached = await AsyncStorage.getItem(CACHE_KEY); if (cached) { const {
data, timestamp } = JSON.parse(cached); if (Date.now() - timestamp < CACHE_DURATION) {
return data as Camera[]; } } // Önbellek yoksa veya eskiyse API'den çek const
cameras = await CameraService.getNearbyCameras(lat, lng, radiusKm); // Önbelleğe kaydet
await AsyncStorage.setItem(CACHE_KEY, JSON.stringify({ data: cameras, timestamp: Date.now()
})); return cameras; }, staleTime: 5 * 60 * 1000, // 5 dakika gcTime: 30 * 60 * 1000,
// 30 dakika });}
```

6.2 Cluster ve Marker Optimizasyonu

Harita üzerinde binlerce kamera gösterirken performansı korumak için cluster (kümeleme) tekniğini kullanın. React Native Maps kütüphanesi ile marker clustering desteği mevcuttur. Yakın kameraları gruplandırarak ve sadece kullanıcı yakınlıktırdığında detayları göstererek uygulama performansını artırabilirsiniz.

7. Yasal Konular ve Uyarılar

Trafik kameraları uygulaması geliştirirken dikkat edilmesi gereken önemli yasal konular bulunmaktadır. Bu konular, uygulamanızın farklı ülkelerde yasal olarak yayınlanabilmesi için kritik öneme sahiptir ve ihlali ciddi yaptırımlara yol açabilir.

7.1 ülke Bazlı Yasaklar

Bazı ülkelerde hız kamerası uyarı uygulamaları yasaktır veya kısıtlıdır. Bu ülkeler arasında Fransa, İsviçre ve bazı Avrupa ülkeleri öne çıkmaktadır. Uygulamanızın bu ülkelerde yasal olarak yayınlanabilmesi için aşağıdaki önlemleri almanız gerekmektedir:

1. Fransa: Hız kamerası konumlarını göstermek yasaktır, "tehlikeli bölge" olarak işaretleyin
2. İsviçre: Hız kamerası uyarı sistemleri tamamen yasaktır, App Store'da erişimi engelleyin
3. Almanya: Yasalıdır ancak kullanıcı onayı gereklidir
4. Türkiye: Yasalıdır, herhangi bir kısıtlama yoktur

7.2 App Store Gereksinimleri

Apple App Store ve Google Play Store, hız kamerası uygulamaları için özel kurallara sahiptir. Uygulamanızın mağazalardan kaldırılmaması için bu kurallara uymanız gerekmektedir:

1. Kullanıcıya açık bir şekilde veri kaynağını belirtin
2. Coğrafi kısıtlamaları uygulayın (ülke bazlı filtreleme)
3. Gizlilik politikası ve kullanım koşulları yayınlayın

4. Kullanıcı verilerini koruyun (GDPR uyumluluğu)

8. Özet ve Öneriler

Bu kılavuzda, dünya genelinde trafik kameraları ve hız tuzakları verilerini ücretsiz olarak toplamak için kapsamlı bir yol haritası sunulmuştur. OpenStreetMap, Lufop ve hükümet açık veri portalları gibi kaynaklardan veri çekebilirsiniz. Supabase ile güçlü bir backend oluşturarak, Expo/React Native uygulamanızda bu verileri verimli bir şekilde kullanabilirsiniz.

Önerilen Adımlar:

- Öncelikle OSM verilerini kullanarak temel veritabanını oluşturun
- Lufop ile Avrupa verilerini zenginleştirin
- Hedef ülkelerin resmi veri portallarını entegre edin
- Kullanıcı raporları ile verileri güncel tutun
- Yasal gereksinimlere mutlaka uyun

Bu kaynakları birleştirerek, dünya genelinde kapsamlı ve güncel bir trafik kameraları veritabanı oluşturabilirsiniz. Başarılar dileriz!

Faydalı Bağlantılar

- OpenStreetMap Wiki: https://wiki.openstreetmap.org/wiki/Tag:highway=speed_camera
- Lufop API: <https://lufop.net/en/lufop-api>
- Supabase Dokümantasyon: <https://supabase.com/docs>
- Expo Dokümantasyon: <https://docs.expo.dev>